SOP: Sentinel Workspace desktop app

Nov 12, 2026 · @Adam Ring · App version 2.3.0

1. Purpose and scope

This SOP is the standard way to use the Sentinel Workspace desktop app: sign in to a client's Microsoft Sentinel workspace, review its live analytics rules, compare them with the team's detection library, and send rule changes through review. How to write a rule in the Rule builder is covered step by step in the companion **SOP: Building detection rules**.

It applies to security engineers and analysts who work in client tenants and contribute to the private detection repository (isi-security/detection-as-code).

The app has three pages, opened from the left sidebar (Ctrl+1, Ctrl+2, Ctrl+3):

Page	Use it to	Needs Azure sign-in
Connect	Sign in to a client tenant and pick the subscription and Log Analytics workspace; set the rule library	Yes
Rules > Workspace rules	Read every analytics rule in the selected workspace and export them	Yes
Rules > Rule library	Browse clients, rules and assignments in the detection repository; assign, preview and deploy	No (GitHub only)
Rules > Drift	Compare what the library says the client should have with what is live in its workspace	Yes, plus the rule library
Rule builder	Create or edit a rule, test its query and submit it for a teammate's approval	No (GitHub only); Run test query needs a workspace

Out of scope: Sentinel audit evidence exports and commercial (Lighthouse) client onboarding are not in the desktop app; Lighthouse onboarding runs from the command line (`py -m sentinel_app.lighthouse_onboarding --help`). GCC High onboarding is held back by policy and appears only when an administrator turns it on.

The app never deploys a rule directly. Every change is a pull request in the detection repository, and the repository's pipeline deploys it after a teammate approves and merges it. The app never asks for or stores passwords: Microsoft handles sign-in and MFA, and GitHub CLI holds the GitHub token.

2. Prerequisites and access

Confirm every item before starting; missing software or access causes most failed sign-ins and reviews. **Check this PC** at the foot of the sidebar checks the software items for you.

Requirement	Detail	How to check
Windows PC	The app is a Windows desktop app	—
Azure CLI 2.76 or newer	Handles Microsoft sign-in and Azure reads	`az --version` or Check this PC
GitHub CLI	Needed for the rule library and for submitting rules	`gh --version` or Check this PC
GitHub sign-in	An account with write access to isi-security/detection-as-code	`gh auth status --hostname github.com`
Client Azure account	Authorized for the client tenant, with read access to the subscription and workspace	Sign in through Connect
Log Analytics Reader or Microsoft Sentinel Reader	On the workspace's resource group; needed for Run test query	Run test query explains a denial
Just-in-time access	Activated through CyberQP or your organization's normal process, if required	CyberQP portal

Requirement	Detail	How to check
Python 3.10+ with Tkinter	Maintainers only: to build the exe or run from source	`py --version`
Local clone of the detection repository	Optional, for the Folder on this PC library source	—

3. One-time setup

Do this once per PC; later sessions start at Procedure 1. Analysts use the packaged app, **Sentinel Workspace.exe**, and do not need Python, Git or the repository.

1. Get **Sentinel Workspace 2.3.0.zip** from the team share and extract it to a folder you can write to, for example `Documents\Sentinel Workspace` (not Program Files). Keep its contents together: the exe, the `desktop-data` folder beside it (client list and rule library setting already included), READ ME FIRST.txt and the SOP PDFs.
   - Upgrading? Replace only `Sentinel Workspace.exe` and keep your own `desktop-data` folder, which holds your drafts and submissions.
2. Install Azure CLI and GitHub CLI once. The exe uses them but does not contain them. If installing is blocked, ask IT. In Terminal run each command, then close and reopen Terminal:
   - `winget install -e --id Microsoft.AzureCLI` (or https://aka.ms/installazurecliwindows)
   - `winget install -e --id GitHub.cli` (or https://cli.github.com)
3. Double-click **Sentinel Workspace.exe**. If Windows shows *Windows protected your PC*, select **More info**, then **Run anyway**. Only one copy runs at a time; opening it again brings the open window forward. **F1** opens the in-app guide at any time.
4. Select **Check this PC** at the foot of the sidebar. Every line should say **OK**; for any line marked **Needs action**, do what it says, reopen the app and select **Check again**.
5. Optional: on the App version line of Check this PC, select **Set update folder…** and enter the team share folder. The app then shows a notice at the top of Connect when a newer version is there. It never downloads or installs anything itself.
6. On **Connect**, in the **Rule library** card, confirm Organization **isi-security** and Repository **detection-as-code**, then **Save**.

7. Select **Sign in to GitHub** if GitHub is not signed in. A one-time code appears in Session activity at the bottom of the page; enter it in the browser window that opens. The card, and the pill in the Rule builder toolbar, then show **GitHub: <your account>**.
8. Add any client not already in the list: in the Sign in card select **+ New client…**, enter the **Client name** and, optionally, the **Tenant ID** or verified domain, then **Add client**. An invalid tenant is rejected before anything is saved.

Profiles and settings stay on this PC in `desktop-data` (sidebar > **Open app data folder**). They never contain credentials.

Building a new version (maintainers only). Install Python 3.12 (tick *Add python.exe to PATH*), Git, Azure CLI and GitHub CLI. Clone `https://github.com/aring87/sentinel-workspace.git` and double-click **Build-Exe.cmd** in a clean checkout of a commit. It installs PyInstaller and `requirements.txt`, draws the icon and version resource, and produces `dist\Sentinel Workspace.exe`. To run from source instead: `py -m pip install -r requirements.txt`, then `py -m sentinel_app`.

Preparing the analyst package (maintainers only). Make a folder named `Sentinel Workspace` holding the new exe, READ ME FIRST.txt, both SOPs as PDF, and a `desktop-data` folder with only:

- `clients.json`, copied from your own desktop-data folder;
- `settings.json` holding only `{"github_owner": "isi-security", "github_repo": "detection-as-code"}`;
- optionally `branding\`, the organization logo for the foot of the sidebar, made with `py scripts\make_logo_sizes.py <logo.png> "Sentinel Workspace\desktop-data\branding"` from one large PNG meant for dark backgrounds.

Zip the folder and put it on the team share, with a `latest.json` beside it, for example `{"version": "2.3.0", "notes": "KQL editor with completions and one-click fixes."}`. Never include the rest of your desktop-data: `sessions` holds your Azure sign-in sessions, `review-requests` holds private rule content, and `access-history.json` and `runs` are your own records. `clients.json` has client names and tenant IDs (no credentials); share it only through approved channels.

4. Procedure 1 — Connect to a client workspace

The outcome is a selected workspace: the header at the top right of every page shows a green dot with the client and workspace (for example *Acme Corporation · law-acme-sentinel*), the window title starts with the client's name, and the Workspace card shows **Connected**.

1. If the client requires just-in-time access, use **Need temporary access?** at the foot of the Sign in card: select **Open CyberQP** and activate access there. Opening CyberQP does not sign the app in.
2. In **Sign in > Client**, choose the client (type its first letters to jump to it). Its saved tenant shows beneath; select **Change** to edit it. Leave the tenant blank only to discover every directory your account can reach.
3. Choose the **Sign-in method**: **Browser** (default), or **Windows account** where the organization requires the Windows broker.
4. Select **Sign in to <client>** (or press Enter in the tenant field). Complete sign-in and MFA with the client-authorized account; choose **Use another account** if the wrong one is suggested. Azure Government (GCC High) tenants are detected and signed in to automatically.
5. While Microsoft waits, a blue panel says where to finish and offers **Bring window forward** and **Cancel sign-in** (use it if you closed the browser tab). Sign-in times out after five minutes. If it fails, a red banner explains why; **Copy details** copies Microsoft's full message for a ticket.
6. After sign-in the app selects the subscription and workspace you used last for this client, or the default subscription and its first workspace. Check the **Subscription** and change it if needed; missing one? Select **↻ Refresh** (F5) or **Can't see a subscription?** and paste the ID from the Azure portal for a read-only report.
7. Choose the **Log Analytics workspace**.
8. **Verify the destination before any work.** Select **+ Show workspace IDs** on the Workspace card to list the Tenant ID, Subscription ID, Resource group, Workspace name and Workspace ID, and confirm they match the client and environment you intend. Right-click an ID to copy it, or **Copy details** for all of them.

Switching clients: choose the other client and sign in to it. Each Microsoft session belongs to the client it was started for and is never used for another. Switching back returns to that client's last workspace without another sign-in. Signing in to the Azure portal in your browser does not sign this app in.

5. Procedure 2 — Review live analytics rules

Workspace rules is a read-only view of what is actually configured in Sentinel, including disabled rules; it never changes a rule.

1. Complete Procedure 1, then select **View analytics rules** on the Workspace card (or **Rules** in the sidebar, Ctrl+2). Rules load automatically; **Refresh** (F5) reads Azure again.
2. Read the totals above the list: **All**, **Enabled**, **Disabled** and **Not specified** (no explicit enabled flag — not the same as disabled).
3. Narrow the list with the search box (Ctrl+F; name, rule ID, severity or type) and the state filter. Select a column heading to sort.
4. Select a rule (or use the arrow keys) to see its details beside the list: state, severity, description, type, schedule, threshold, tactics, techniques, last modified, rule ID, the KQL query (**Copy query**) and the raw configuration. Right-click a value to copy it.
5. To keep a record, select **Export all** and choose approved client storage. The ZIP holds raw JSON, CSV and collection details for every loaded rule, regardless of the search or filter.

If the summary says **Incomplete inventory**, some rules could not be read (usually a permission gap). The export then asks you to confirm and is labelled incomplete; treat it as partial evidence only.

6. Procedure 3 — Browse the rule library and check drift

The library shows what the detection repository says should be deployed; it is configuration only, not live Sentinel state. No Azure sign-in is needed.

1. Open **Rules** and choose **Rule library** at the top.
2. Choose the source:
   - **GitHub** — reads isi-security/detection-as-code from its default branch at one commit. Required for assigning, previewing and deploying.
   - **Folder on this PC** — **Choose folder…** and pick your clone's root (it must contain `clients/` and `rules/sentinel/`). Shows working files, including uncommitted changes.
3. Select **Refresh library** (F5). The GitHub read takes about 15 seconds; progress shows while it runs. The Library source card then shows the repository, when it was loaded and the signed-in account.

4. Switch between **Rules** and **Clients**, search (Ctrl+F) and filter by configured state. Select a row to see its assignments and raw configuration; double-click a rule, or **Edit selected rule**, to open it in the Rule builder.
5. Check the pill on the Rule library card. If it shows issues, open the Library ··· menu > **Library issues** and fix them in the repository first — reviews and assignments are blocked while issues exist.

Where	Action	What it does
Above the list, rule selected	Edit selected rule	Opens the rule in the Rule builder
Above the list, rule selected	Assign to clients…	Prepares an assignment for a teammate's approval (Procedure 5)
Above the list, rule selected	Preview and deploy…	Starts a client preview through the repository workflows (Procedure 5)
Library ··· menu	Library issues	Lists unreadable or invalid files
Library ··· menu	Open saved assignment…	Reopens a saved assignment request
Library ··· menu	Preview & deployment history…	Lists saved preview and deployment requests with their GitHub status

Check drift between the library and the workspace

Drift compares what the library says the connected client should have with what is live in its workspace. It uses what Workspace rules and Rule library have already read and changes nothing.

1. Complete Procedure 1, open **Rules** and choose **Drift**. If the library isn't loaded, the app reads it from GitHub once.
2. The app finds the library target that deploys to the selected workspace (by workspace ID, or by subscription, resource group and workspace name). If none or several match, the list says so; check the client's target files.
3. Read the three kinds of drift: **Missing** (assigned but not in the workspace), **Different** (enabled state or severity differs, after the target's overrides) and **Only in workspace** (live rules the library doesn't assign). Other settings, such as the schedule, don't count.

4. Select a rule to see the library's and the workspace's settings side by side, differences in red. Right-click to copy its name or ID, or open it in Workspace rules or Rule library.
5. Resolve drift through the library and the reviewed process (Procedures 4 and 5), never by editing the live rule in the portal.

7. Procedure 4 — Create, edit and submit a rule

Rules are written in the **Rule builder** (Ctrl+3) and reach clients only through a reviewed pull request. The full field-by-field procedure, the KQL editor and the validation rules are in **SOP: Building detection rules**; this is the outline.

1. Start with **New rule** (Ctrl+N), **Edit existing rule…** (opens the library to pick from) or **Open…** (recent drafts, or a rule file from disk).
2. Work through the five steps: **Basics**, **Query & schedule**, **MITRE ATT&CK**, **Advanced**, **Review**. Fix anything the KQL check underlines, and select **Run test query** to try the query read-only against the connected workspace.
3. **Save draft** (Ctrl+S) as you go. Unsaved work is also autosaved every 30 seconds and offered back after a crash.
4. Select **Check rule** until it passes, then **Submit for approval**. Read the changes, fill in **Why is this changing?** and submit. The app opens a ready-for-review pull request; a new rule's pull request also lists it in the deploy workflows' rule dropdowns.
5. Ask a teammate to approve it, and follow it in **My submissions…** until **Merged**.

New rules start disabled. Never edit a rule in the Sentinel portal or on the repository's main branch to skip review.

8. Procedure 5 — Assign, preview and deploy a rule

A merged rule reaches a client in three reviewed moves: assign it to the client's workspace target, preview it, then deploy it. All three need the **GitHub** library source with write access, and none of them can skip the repository's own approvals or Azure permissions.

Assign to clients

1. In **Rules > Rule library** (GitHub source, issues resolved), select the rule and choose **Assign to clients…**.

2. Search by client, workspace or target and click rows (or press Space) to select workspaces; **Select shown** adds the visible rows. Up to 100 workspaces per request.
3. Choose **Enabled** or **Disabled** for the rule in those workspaces. **Disabled** is the default; other overrides stay as they are.
4. Select **Check** and review the repository, account, revision, destinations and the YAML changes. This only reads GitHub.
5. Select **Submit for approval**. All changed client manifests go into one pull request; **Open in GitHub** to follow review and CI. After it merges, **Refresh library**.

Assigning adds assignments or changes the enabled override; it never removes an assignment. If submission is interrupted, use **Library ⋯ > Open saved assignment…** to resume it rather than starting a new one.

Preview, then deploy

1. Select the rule and choose **Preview and deploy…**. Select assigned, enabled client targets (**Select all eligible** picks every target that already references the rule).
2. Select **Prepare request** and check the exact clients, rule, revision and number of batches. Nothing has started yet.
3. Select **Start previews**. This runs the repository workflow in preview mode on `main`. **Refresh status** reads each run; **Open GitHub** shows its checks and what-if artifacts.
4. When every preview run has finished successfully at the reviewed revision, **Deploy…** becomes available. Review the repository, account, rule, workspaces, enabled state, overrides and preview runs.
5. Select **Start deployment**. **Refresh status** and **Open GitHub** follow each run and any environment approval. A submitted run is not a successful deployment until GitHub says so.

If GitHub times out, a run may still have started: check **Library ⋯ > Preview & deployment history…** and the repository's Actions page before preparing another request. A newer `main` revision needs a refreshed library and new previews.

9. Refreshing access, Check this PC and support

None of these change roles or resources.

Action	Where	Use when
↻ Refresh (subscriptions)	Beside the Subscription list	A subscription was just granted or JIT access just started
↻ Refresh (workspaces)	Beside the workspace list	A workspace was just created or access just granted
Can't see a subscription?	Under the Subscription list	The portal shows a subscription the app doesn't; paste its ID for a read-only report
Check this PC	Foot of the sidebar	Something isn't working: checks Azure CLI, GitHub CLI, GitHub sign-in, the data folder and the app version
Copy support info	Sidebar, or foot of Check this PC	Opening a ticket: versions, check results and recent errors, never client names, IDs, accounts, tokens or rules
Sign out	Account panel, foot of the sidebar	End of work; clears this app's Azure session

If Microsoft asks for MFA again mid-session, the app asks **Complete Microsoft authentication?**: select **Open sign-in**, finish signing in, then start the operation again (it is never replayed automatically). Contributor grants and Azure access management are onboarding tasks, not in the app — ask an engineer.

10. Troubleshooting

Start with **Session activity** at the bottom of Connect: it holds the full text of anything the status bar shortens. Every failure dialog has **Copy details**. Rule builder problems are covered in SOP: Building detection rules.

Symptom	Likely cause	Action
Microsoft window does not appear	Opened behind other apps	**Bring window forward**, or Alt+Tab

Symptom	Likely cause	Action
Stuck on "Waiting for Microsoft sign-in"	Browser tab closed or abandoned	**Cancel sign-in**, then sign in again
"Complete Microsoft authentication"	MFA or Conditional Access needs a fresh sign-in	**Open sign-in**, sign in, start the operation again
AADSTS50020, 50034 or 51004	Wrong account or tenant	Check the client's tenant and the active CyberQP account; **Use another account**
No subscriptions after sign-in	No access, wrong directory or JIT not active	Activate JIT, **↻ Refresh**, or **Can't see a subscription?**
No workspaces	No read access in that subscription	Pick another subscription or request Log Analytics read access
"Incomplete inventory"	Some rules could not be read	Check Sentinel read permissions; export only with the incomplete label
Library unavailable / GitHub read failed	Not signed in, wrong repository or no org access	`gh auth status`; **Sign in to GitHub**; confirm isi-security/detection-as-code
Review or assignment blocked: library issues	Invalid or unreadable repository files	**Library issues**; fix them in the repository first
Drift: no library target deploys to this workspace	Client target names another workspace, or none	Check the client's target file; ask an engineer
Deploy… stays greyed	A preview failed, is still running or used another revision	**Refresh status**; run new previews after the library changes
Check this PC: Azure CLI or GitHub CLI needs action	Not installed, or Azure CLI older than 2.76	Install or update (Section 3), reopen the app, **Check again**

Symptom	Likely cause	Action
Status bar says "Something went wrong"	An unexpected error in the app	Read Session activity; send **Copy support info** or `desktop-data\logs\errors.log` to the maintainer
Opening the exe again only brings a window forward	Only one copy runs at a time	Use the open window from the taskbar

11. Security, data handling and end of session

Treat everything the app shows or exports as client-confidential.

- Use only accounts authorized for the client, and activate JIT access through the approved process.
- Confirm the workspace IDs (Procedure 1, step 8) before any export, test query or rule change.
- Save rule exports only in approved client storage; the ZIP contains rule queries and configuration.
- Do not paste client identifiers, queries or test-query results into tickets or chats outside approved channels. Copy details puts tenant and subscription IDs on the clipboard.
- Run test query only reads the workspace and keeps no results. Copy support info leaves out client data; still read it before pasting.
- Rule changes go only through a reviewed pull request. Never bypass review, edit `main` directly or change a live rule in the portal.
- Do not disable MFA or Conditional Access to get past a sign-in failure.
- Never share your `desktop-data` folder: it holds your sign-in sessions and private rule content.

End-of-session checklist:

- [ ] Rule drafts saved or deliberately discarded
- [ ] Submissions made and their GitHub links added to the ticket
- [ ] Exports saved to approved client storage
- [ ] **Sign out** selected in the sidebar (closing the app also signs out of its Azure session)
- [ ] JIT access ended in CyberQP, if your process requires it

12. Quick reference

Indicator	Where	Meaning
Ready / Working pill	Status bar	Whether an operation is running; buttons that start work are greyed while Working
Green dot + client · workspace	Header, every page	Your destination; point at it for tenant and subscription IDs
Not connected / Signed in / Connected	Workspace card and sidebar	Azure session state
GitHub: account / not signed in / Rule library not set	Rule library card, Rule builder toolbar	GitHub and library state, with the fix beside it
Update notice	Top of Connect	A newer version is in the update folder; **Open folder** shows it

Item	Location
Clients and settings	`desktop-data\` (sidebar > Open app data folder)
Rule drafts and recovery copy	`desktop-data\drafts\`
My submissions, assignments, previews	`desktop-data\review-requests\`, `assignment-requests\`, `preview-requests\`
Error log	`desktop-data\logs\errors.log`
Window layout, last workspace, editor theme	`desktop-data\ui-state.json`
Detection repository	isi-security/detection-as-code (rules in `rules/sentinel/`, clients in `clients/`)
In-app guide	F1, or sidebar > Help & prerequisites

Keyboard shortcuts:

- **F1** guide · **Ctrl+1 / 2 / 3** Connect, Rules, Rule builder · **F5** refresh the current page · **Ctrl+F** search
- **Ctrl+N** new rule · **Ctrl+S** save draft · **Ctrl+Page Down / Up** next or previous builder step
- In lists: arrow keys move, **Enter** opens the selected rule, **Ctrl+C** copies rows
- In dialogs: **Esc** closes, **Enter** presses the blue button (never a deployment)